

Documentation Arduino LMS portable v1.1

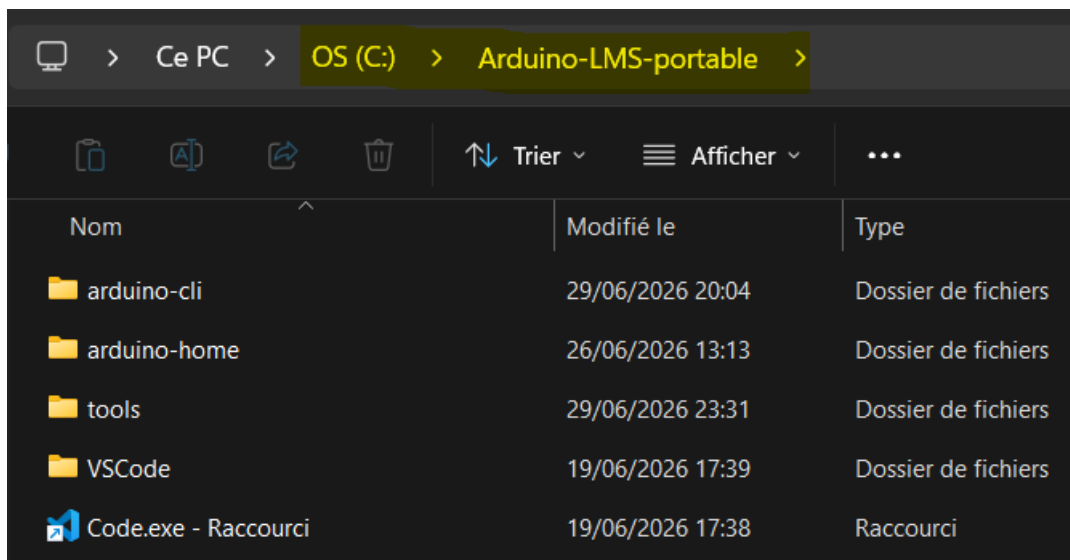
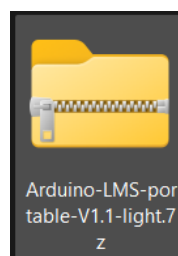
Juillet 2026

Table des matières

Documentation Arduino LMS portable v1.1.....	1
Installation du pack	1
Créer un projet.....	2
Compiler un projet.....	4
Configurer le port COM (obligatoire avant Upload)	5
Téléverser le binaire du projet (ou <i>Upload</i>).....	6
Pour aller plus loin	7
Installer une librairie	7
Installer un <i>core</i>	8

Installation du pack

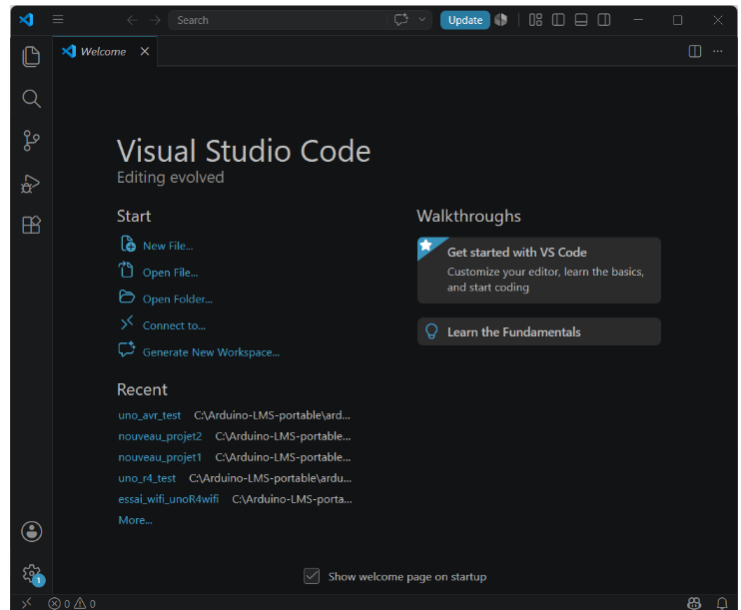
- Archive .7z du pack à récupérer depuis les *Releases* sur Github :
<https://github.com/fleb72/Arduino-LMS-portable/releases>
 - Avec 7zip (gratuit), extraire l'archive sur le disque C : \
- Vous devez avoir exactement :



Et le pack est prêt !! Aucune installation à faire, ce dossier `C: /Arduino-LMS-portable` (environ 2,3 Go) est **portable**. Si vous voulez déployer l'environnement sur d'autres PC → copier-coller du dossier sur le disque C:/ du PC de destination, c'est tout !

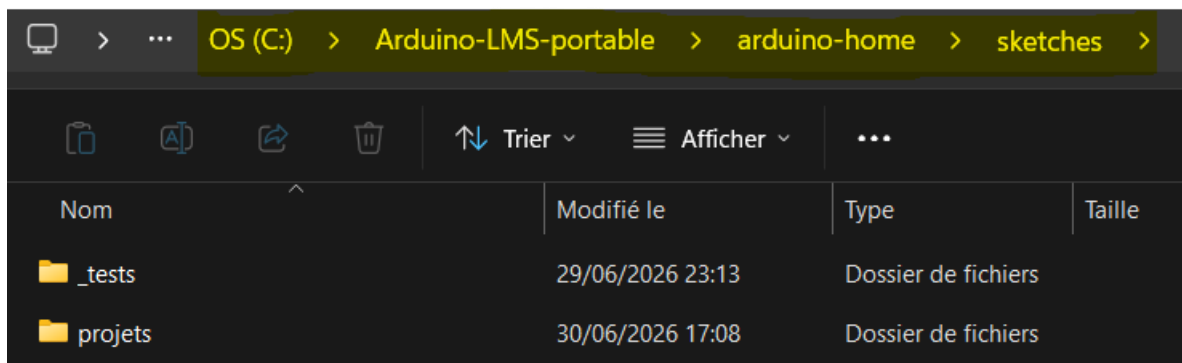
Pas besoin de droits admin, on a un (gros) dossier qui contient tout : les sketches, les *cores* des cartes, les bibliothèques, les outils de compilation et de téléversement, tout l'environnement...

- Pour démarrer l'environnement, un double-clic sur le raccourci `Code.exe` du dossier, et la fenêtre VS Code s'ouvre :



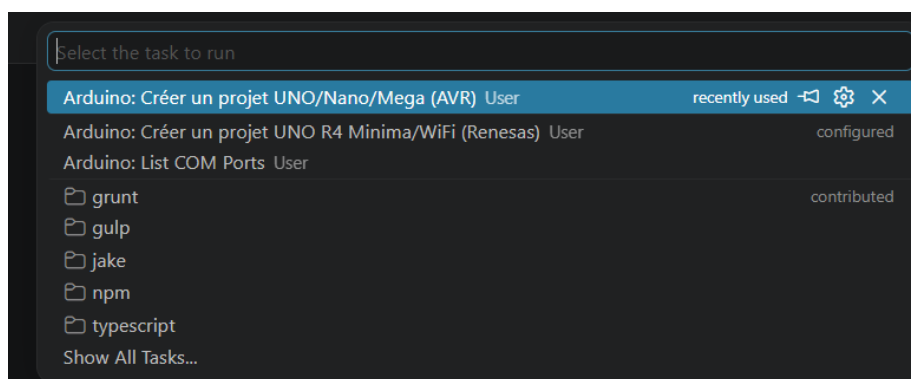
Créer un projet

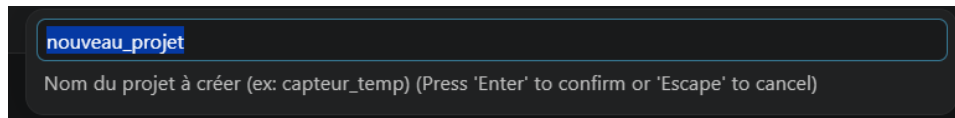
Les sketches sont dans `C:\Arduino-LMS-portable\arduino-home\sketches` :



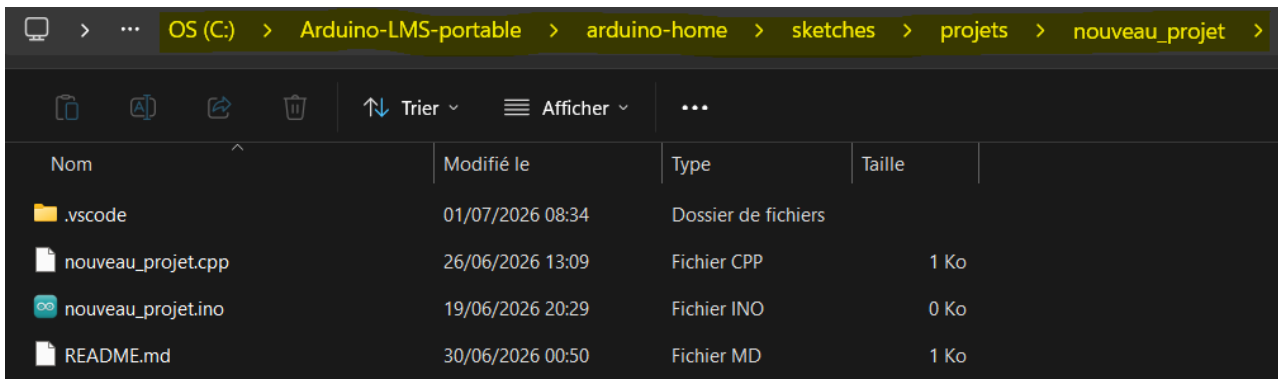
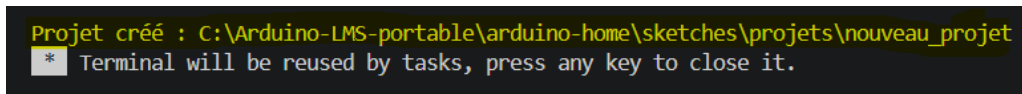
Les sketches dans le sous-dossier `_tests` peuvent servir de *templates* pour de nouveaux projets. Par exemple pour un projet Arduino Uno, vous pouvez recopier le dossier `_tests\uno_avr_test` dans `projets`, renommer le dossier, et les fichiers `.ino` et `.cpp`.

Mais le plus simple, est de passer par VS Code avec le menu `Terminal -> Run Task... -> Arduino : créer un projet UNO/Nano/Mega (AVR)` et de renseigner le nom du projet :



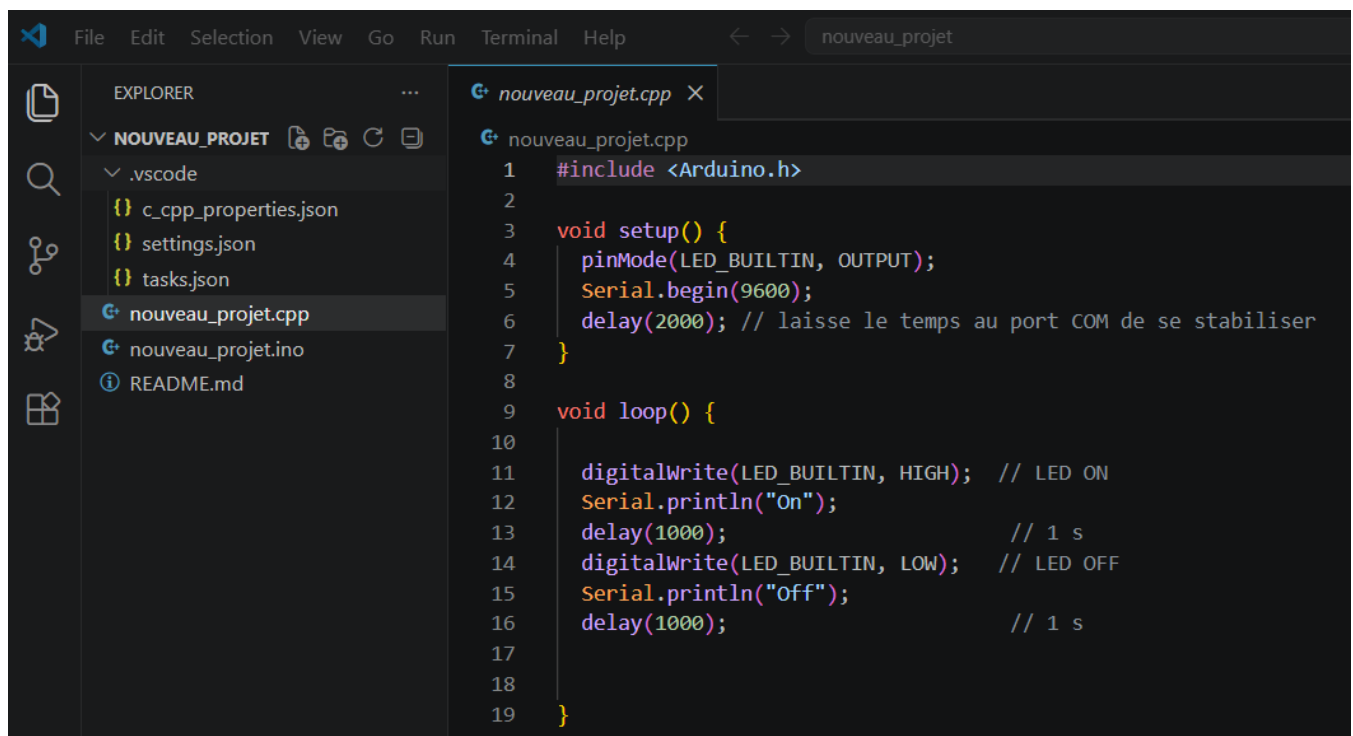


Après validation, le dossier du projet est automatiquement créé dans `C:\Arduino-LMS-portable\arduino-home\sketches\projets\<Nom du projet>`:



- Le code du projet est dans le fichier `.cpp` (ici, un simple clignotement de led) ;
- Le fichier `.ino` est vide, mais sa présence est obligatoire pour VS Code ;
- Le sous-dossier `.vscode` contient des fichiers de configuration, on y reviendra plus loin.

Pour travailler le sketch dans VS Code, menu `File -> Open Folder...` :



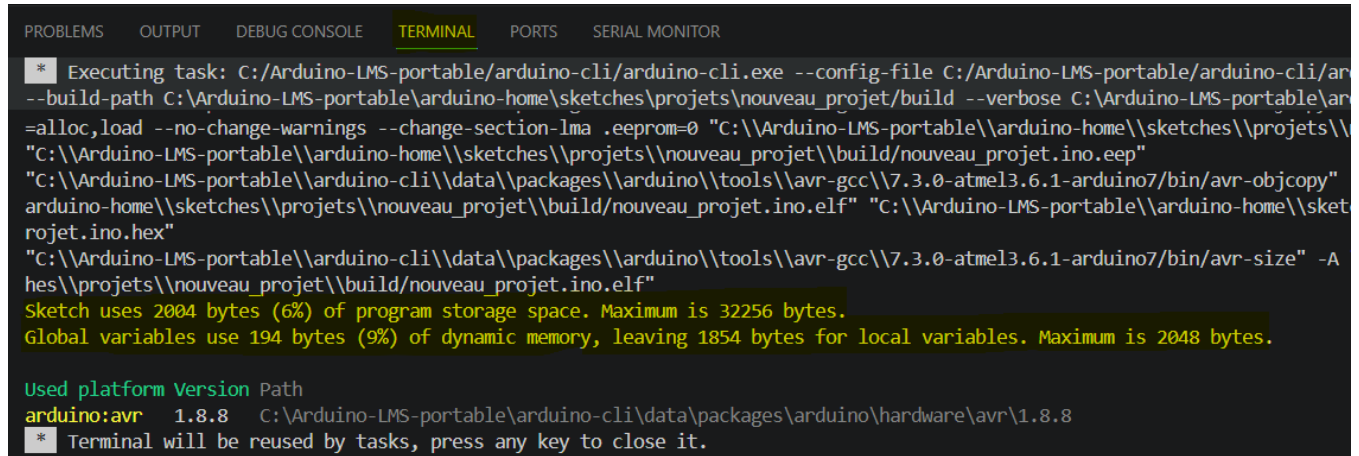
Ce projet est déjà prêt pour compilation et téléversement.

Compiler un projet

Deux façons de faire :

- soit par le menu **Terminal -> Run Build Task...** (raccourci Ctrl+Shift+B) ;
- soit par le menu **Terminal -> Run Task...**, puis sélectionnez la tâche **Arduino : Build**.

Le Terminal intégré à VS Code s'ouvre et rend compte :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

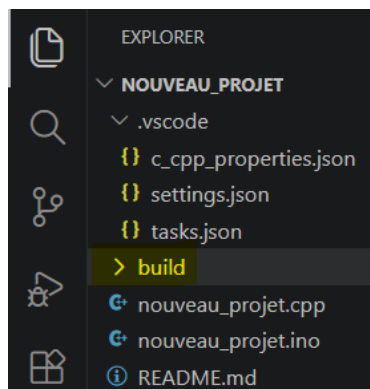
[*] Executing task: C:/Arduino-LMS-portable/arduino-cli/arduino-cli.exe --config-file C:/Arduino-LMS-portable/arduino-cli/arduino-cli.conf --build-path C:\Arduino-LMS-portable\arduino-home\sketches\projets\nouveau_projet\build --verbose C:\Arduino-LMS-portable\arduino-cli\data\packages\arduino\tools\avr-gcc\7.3.0-atmel3.6.1-arduino7/bin/avr-objcopy --no-change-warnings --change-section-lma .eeprom=0 "C:\Arduino-LMS-portable\arduino-home\sketches\projets\nouveau_projet\build\nouveau_projet.ino.eep" "C:\Arduino-LMS-portable\arduino-home\sketches\projets\nouveau_projet\build\nouveau_projet.ino.elf" "C:\Arduino-LMS-portable\arduino-home\sketches\projets\nouveau_projet\build\nouveau_projet.ino.hex"
"C:\Arduino-LMS-portable\arduino-cli\data\packages\arduino\tools\avr-gcc\7.3.0-atmel3.6.1-arduino7/bin/avr-size" -A C:\Arduino-LMS-portable\arduino-home\sketches\projets\nouveau_projet\build\nouveau_projet.ino.elf
Sketch uses 2004 bytes (6%) of program storage space. Maximum is 32256 bytes.
Global variables use 194 bytes (9%) of dynamic memory, leaving 1854 bytes for local variables. Maximum is 2048 bytes.

Used platform Version Path
arduino:avr 1.8.8 C:\Arduino-LMS-portable\arduino-cli\data\packages\arduino\hardware\avr\1.8.8

[*] Terminal will be reused by tasks, press any key to close it.
```

Ici, la compilation s'est déroulée sans erreurs...

Un nouveau dossier **build** apparaît dans la structure du projet :

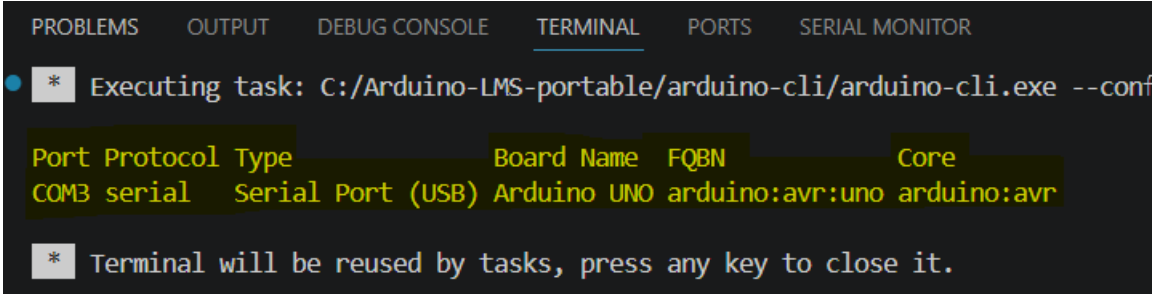


Ce dossier contient des données en cache, et qui pourront être reprises pour accélérer les compilations suivantes → conserver ce **build**.

Configurer le port COM (obligatoire avant Upload)

Branchez votre carte Arduino sur le port USB, puis allez dans le menu :

Terminal -> Run Task... -> Arduino : list COM ports



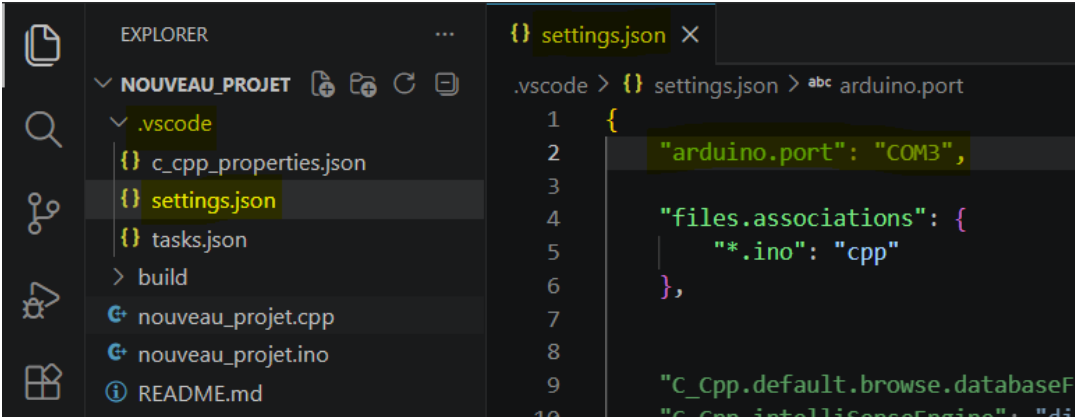
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
* Executing task: C:/Arduino-LMS-portable/arduino-cli/arduino-cli.exe --conf

Port Protocol Type Board Name FQBN Core
COM3 serial Serial Port (USB) Arduino UNO arduino:avr:uno arduino:avr

* Terminal will be reused by tasks, press any key to close it.
```

Dans mon cas, l'Arduino UNO est bien reconnue sur le port COM 3.

Il faut renseigner la ligne `arduino.port` du fichier `.vscode/settings.json` :



```
EXPLORER
NOUVEAU_PROJET
  .vscode
    c_cpp_properties.json
    settings.json
    tasks.json
  build
  nouveau_projet.cpp
  nouveau_projet.ino
  README.md

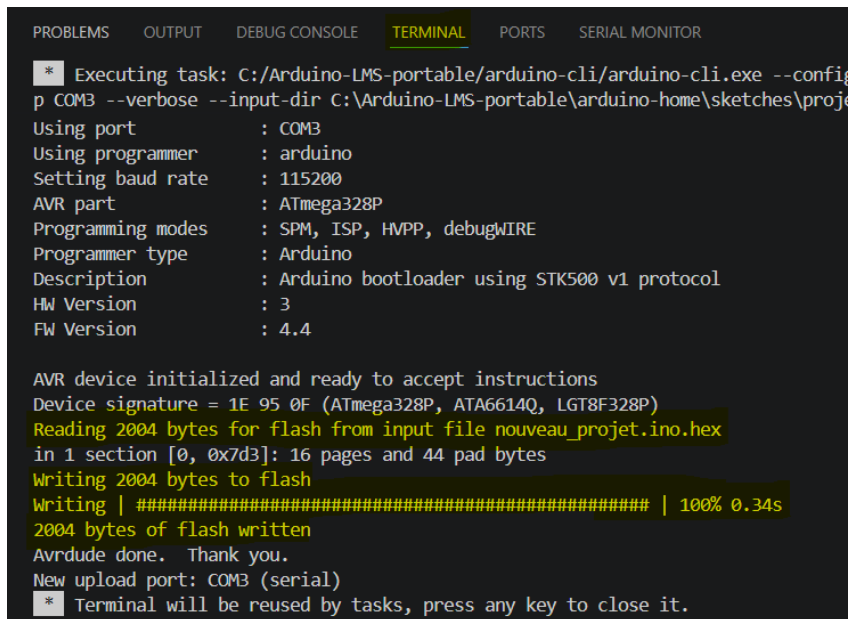
settings.json
.vscode > {} settings.json > abc arduino.port
1 {
2   "arduino.port": "COM3",
3
4   "files.associations": {
5     "*.ino": "cpp"
6   },
7
8   "C_Cpp.default.browse.databaseFi
9   "C_Cpp.intelliSenseEngine": "di
```

Dans mon cas : `"arduino.port": "COM3"`, et sauvegardez les modifications (ctrl+S).

Téléverser le binaire du projet (ou *Upload*)

Allez dans le menu **Terminal -> Run Task... -> Arduino : Upload**.

Le Terminal s'ouvre et rend compte de l'Upload :



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SERIAL MONITOR

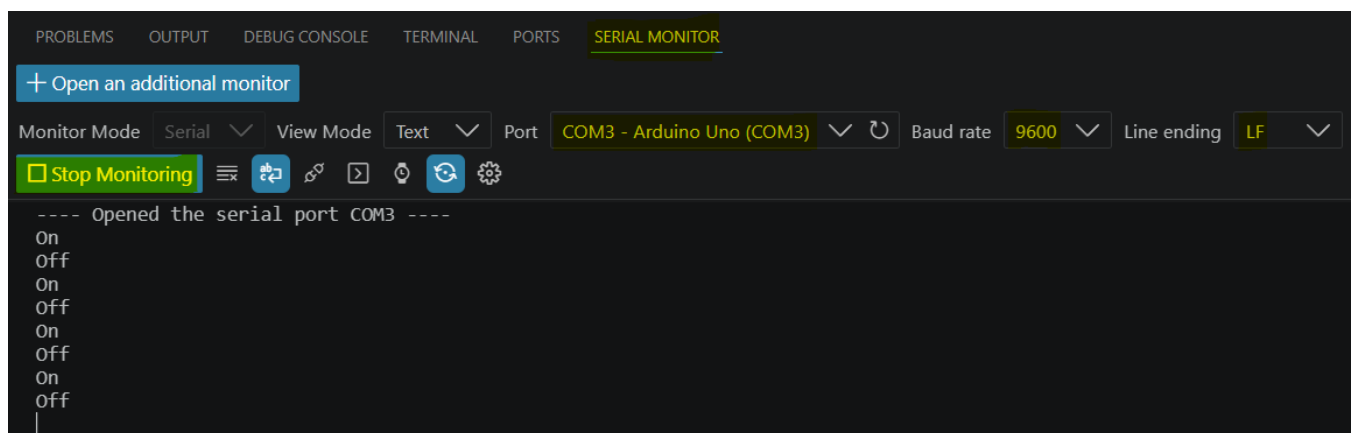
[*] Executing task: C:/Arduino-LMS-portable/arduino-cli/arduino-cli.exe --config
p COM3 --verbose --input-dir C:\Arduino-LMS-portable\arduino-home\sketches\proj
Using port          : COM3
Using programmer    : arduino
Setting baud rate   : 115200
AVR part            : ATmega328P
Programming modes   : SPM, ISP, HVPP, debugWIRE
Programmer type     : Arduino
Description         : Arduino bootloader using STK500 v1 protocol
HW Version          : 3
FW Version          : 4.4

AVR device initialized and ready to accept instructions
Device signature = 1E 95 0F (ATmega328P, ATA6614Q, LGT8F328P)
Reading 2004 bytes for flash from input file nouveau_projet.ino.hex
in 1 section [0, 0x7d3]: 16 pages and 44 pad bytes
Writing 2004 bytes to flash
Writing | ##### | 100% 0.34s
2004 bytes of flash written
Avrdude done. Thank you.
New upload port: COM3 (serial)
[*] Terminal will be reused by tasks, press any key to close it.
```

La Led en surface de la carte devrait clignoter...

A noter que vous pouvez compiler et téléverser dans la foulée en une seule tâche lancée depuis le menu : **Terminal -> Run Task... -> Arduino : Build + Upload**.

Comme il y a des `Serial.println()` dans le sketch, vous pouvez voir les données retournées au PC via le câble USB grâce au Terminal Série intégré (ou **SERIAL MONITOR**) :



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SERIAL MONITOR

+ Open an additional monitor

Monitor Mode Serial View Mode Text Port COM3 - Arduino Uno (COM3) Baud rate 9600 Line ending LF

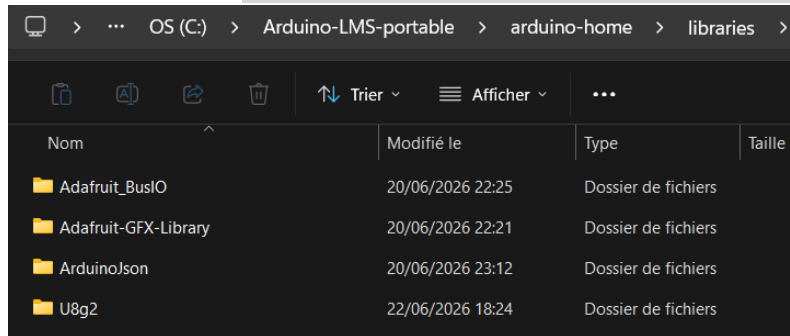
[ ] Stop Monitoring

---- Opened the serial port COM3 ----
On
Off
On
Off
On
Off
On
Off
On
Off
|
```

Pour aller plus loin

Installer une librairie

Les librairies utilisateur sont dans `C:\Arduino-LMS-portable\arduino-home\libraries` :

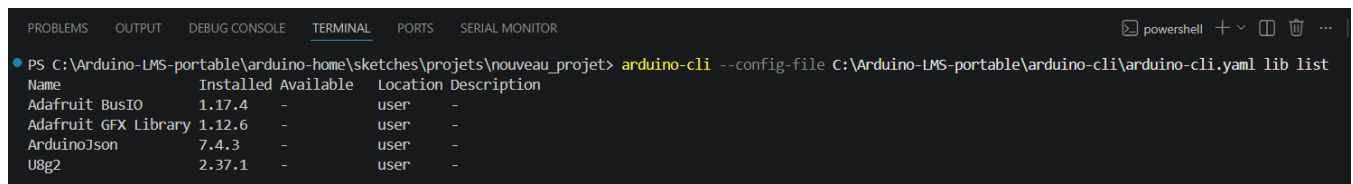


Nom	Modifié le	Type	Taille
Adafruit_BusIO	20/06/2026 22:25	Dossier de fichiers	
Adafruit-GFX-Library	20/06/2026 22:21	Dossier de fichiers	
ArduinoJson	20/06/2026 23:12	Dossier de fichiers	
U8g2	22/06/2026 18:24	Dossier de fichiers	

Comme on souhaitait un pack portable, il n'y en a que très peu à disposition par défaut... et vous n'en avez sans doute pas besoin.

Pour avoir la liste des librairies installées, tapez la commande suivante dans un Terminal intégré à VS Code (éventuellement, menu `Terminal -> New Terminal`) :

```
arduino-cli --config-file C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml lib list
```



Name	Installed	Available	Location	Description
Adafruit BusIO	1.17.4	-	user	-
Adafruit GFX Library	1.12.6	-	user	-
ArduinoJson	7.4.3	-	user	-
U8g2	2.37.1	-	user	-

Par exemple, pour installer la librairie [Servo](#), avec un accès Internet.

Dans le Terminal :

```
arduino-cli --config-file C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml lib install Servo
```

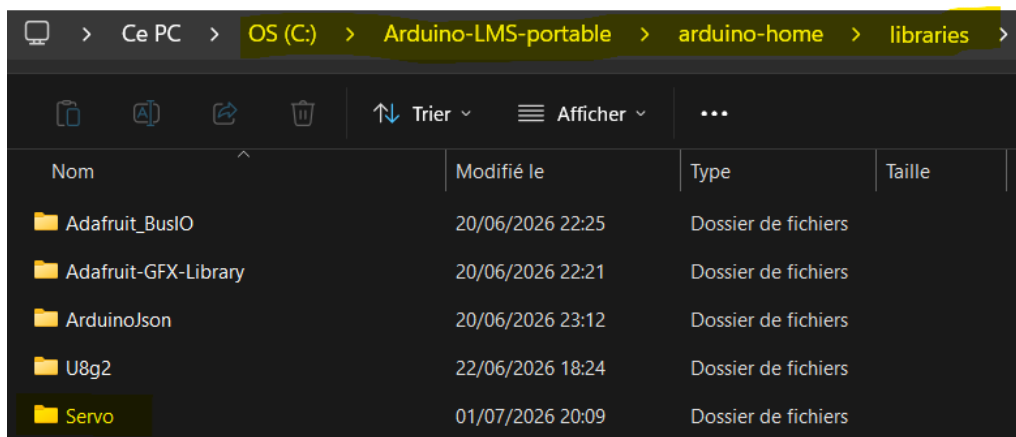
au résultat :

```
Downloading Servo@1.3.0...
```

```
Servo@1.3.0 downloaded
```

```
Installing Servo@1.3.0...
```

```
Installed Servo@1.3.0
```

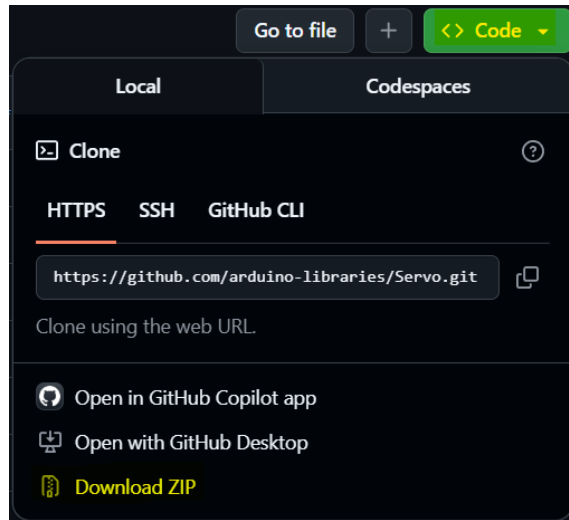


Nom	Modifié le	Type	Taille
Adafruit_BusIO	20/06/2026 22:25	Dossier de fichiers	
Adafruit-GFX-Library	20/06/2026 22:21	Dossier de fichiers	
ArduinoJson	20/06/2026 23:12	Dossier de fichiers	
U8g2	22/06/2026 18:24	Dossier de fichiers	
Servo	01/07/2026 20:09	Dossier de fichiers	

Si le poste n'a pas accès à Internet, il faut récupérer ce dossier `Servo` depuis un autre poste et le coller dans `C:\Arduino-LMS-portable\arduino-home\libraries`.

Les librairies officielles ont un dépôt GitHub, par exemple pour Servo : <https://github.com/arduino-libraries/Servo>.

Cliquez sur le bouton vert `<>Code` et sur `Download ZIP` :



En décompressant le ZIP, vous obtenez un dossier `Servo-master`, que vous renommez en `Servo`.

Il n'y a plus qu'à copier ce dossier dans le pack portable, dans `C:\Arduino-LMS-portable\arduino-home\libraries\`. Et c'est fini !

Installer un core

Un core Arduino est le « pack de support » d'une famille de cartes : il contient tout ce qu'il faut pour compiler et téléverser (compilateur, drivers, définitions de cartes).

Dans le pack Arduino LMS portable, il n'y a que deux cores installés :

```
arduino-cli --config-file C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml core list
```

ID	Installed	Latest	Name
arduino:avr	1.8.8	1.8.8	Arduino AVR Boards
arduino:renesas_uno	1.6.0	1.6.0	Arduino UNO R4 Boards

Et vous pouvez travailler avec les cartes Arduino Uno, Nano, Mega officielles, ainsi que les cartes plus récentes Arduino Uno R4 Minima et WiFi.

Si un jour vous voulez jouer avec un [devKit ESP32](#) de chez Espressif (ou un de ses clones à bas coût) dans l'environnement Arduino, vous devrez installer le core ESP32 pour Arduino... un core monolithique de plus de **5 Go** pour toute la galaxie ESP32 unifiée !? Un monstre...

Avant d'installer le core ESP32, il faut d'abord ajouter l'URL du paquet dans la configuration (`board_manager / additional URLs`).

Complétez le fichier `C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml` :


```
...
board_manager:
  additional_urls:
    - https://resource.heltec.cn/download/package_heltec_esp32_index.json
    - https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

La commande d'installation du *core* est :

```
arduino-cli --config-file C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml core install esp32:esp32
```

... et allez prendre un café.

En fin d'installation, votre pack portable a grossi de 5 à 6 Go !!

Le package ESP32 est dans le dossier `C:\Arduino-LMS-portable\arduino-cli\data\packages\esp32\`.

```
arduino-cli --config-file C:\Arduino-LMS-portable\arduino-cli\arduino-cli.yaml core list
```

ID	Installed	Latest	Name
arduino:avr	1.8.8	1.8.8	Arduino AVR Boards
arduino:renesas_uno	1.6.0	1.6.0	Arduino UNO R4 Boards
esp32:esp32	3.3.10	3.3.10	esp32

Un programme test qui peut servir de *template* pour un futur projet est déjà disponible dans

```
C:\Arduino-LMS-portable\arduino-home\sketches\_tests\esp32_test
```

Cette démo peut maintenant compiler.

Lors de la première compilation, l'outil doit construire l'intégralité du *core* (FreeRTOS, WiFi, Bluetooth, drivers, etc.). Cela peut prendre **jusqu'à 40 minutes** selon la machine (surtout avec les PC à disques mécaniques, très mal lotis pour manipuler les milliers de petits fichiers). Conservez bien le dossier `build` pour les compilations suivantes.